

Remember: this is an oral presentation homework! You are required to work in groups of three, and may collaborate only with other students in your group. We encourage you to write up solutions, but your grade will be based entirely on your presentation. At your presentation, the grader will randomly assign each student to present a problem.

You may use the internet or LLMs to look up formulas, definitions, concepts, etc., but may not simply look up the answers online or ask an LLM.

1 Dumbbell Matching (33 points)

You belong to a gym which has two sets of dumbbells A and B , each of which has n dumbbells. You know that there is a correspondence between the sets: for every dumbbell in set A there is exactly one dumbbell in set B that has the same weight, and similarly for every dumbbell in set B there is exactly one dumbbell in set A that has the same weight. You want to perform exercises that require two dumbbells of the same weight. So you want to pair up the dumbbells by weight, i.e., for every dumbbell you want to know which dumbbell from the other set has the exact same weight.

Unfortunately the dumbbells are unsorted and unlabeled, and you can't tell their weights by looking at them. The only way to compare two dumbbells is to pick them both up simultaneously (one in each hand) and perform a curl. By comparing the strain on your arms, you can tell whether the two dumbbells are the same weight, and if not, which one is heavier. Even more unfortunately, the owner of the gym has a rule that two dumbbells from the same set cannot be used at the same time. So you can compare a dumbbell from set A to a dumbbell from set B , but cannot compare two dumbbells from the same set.

Prove that any deterministic algorithm which solves the problem must make at least $\Omega(n \log n)$ comparisons.

In this problem, we must compare 2 sets of n elements to find each dumbbell's pair. This is a comparison-based

2 Group Sorting (33 points)

In the last homework you designed an algorithm for k -group-sorting in time $O(n \log k)$. Prove a matching lower bound in the comparison model. That is, prove that any algorithm for k -group-sorting in the comparison model must make at least $\Omega(n \log k)$ comparisons in the worst case.

TODO

3 Sorting Different-Length Items (34 points)

We saw in class some fast sorting algorithms where we assumed that the elements were integers with the same number of digits. In this problem we change those assumptions slightly.

Suppose now that we are given an array of strings (over some finite alphabet, say the letters **a-z**). Each string can have a different number of characters, but the total number of characters

in all the strings (i.e., the sum over the strings of the length of the string) is equal to n . Show to sort the strings lexicographically in $O(n)$ time. Here lexicographic order is the standard alphabetic order, so for example $\mathbf{a} < \mathbf{ab} < \mathbf{b}$.

Algorithm Description:

Since the total number of characters in all strings is n , we know that any array A must have at most n elements (in which case every element has 1 character). For this, I consider a tree structure, wherein the A is reconstructed in tree format, with a root node, and each element is represented as characters cascading down a tree. There is a *root* node from which all elements originate, and each ends with a *null* terminator. For each string, I find its place in the tree by cascading down the tree. Each character is a node. If the node exists in the tree, then there is no need to add it again, and the next character is added as a child of that node. If the node does not exist, its place is found by iterating through the children to find at which point the character fits. Since the alphabet has 26 letters, there would be at most 26 comparisons, so this sorting would take $O(26)$ comparisons, or $O(1)$. At the end of the string, a null terminator character would be added to indicate the end of the string, which would be sorted sequentially first. Since there is a constant number of letters in the alphabet, each operation would take $O(1)$ time. This constructs a tree by following this process for each character in each element in A . Since there are n characters spread throughout A , this would be an $O(1)$ process repeating n times, making the time to build the tree $O(n)$.

From here, we must reconstruct the tree to create a sorted list of characters.

Correctness Proof:

TODO

Runtime Proof:

TODO